

NUMU Merchant Hub — Comprehensive Technical Deep-Dive

Document Version: 1.0

Date: June 2025

Audience: Engineers, architects, technical stakeholders

Perspective: Senior Software Engineer — 20 years experience

Table of Contents

1. [Executive Summary](#)
2. [Technology Stack](#)
3. [Project Structure & Architecture](#)
4. [Build System & Developer Experience](#)
5. [Application Entry Point & Initialization](#)
6. [Routing & Navigation Guards](#)
7. [State Management — Context Providers](#)
 - 7.1 [AuthContext — JWT Authentication](#)
 - 7.2 [StoreContext — Multi-Store Management](#)
 - 7.3 [LanguageContext — i18n & RTL](#)
8. [API Layer — Service Architecture](#)
 - 8.1 [Base API Client](#)
 - 8.2 [Auth Service](#)
 - 8.3 [Store Service](#)
 - 8.4 [Product Service & Adapter Pattern](#)
 - 8.5 [Order Service](#)
 - 8.6 [Customer Service](#)
 - 8.7 [Analytics Service](#)
 - 8.8 [Category Service](#)
 - 8.9 [Coupon Service](#)
 - 8.10 [Theme Service & V2 Section Engine](#)
9. [UI Component Library](#)
10. [Layout System](#)
 - 10.1 [DashboardLayout](#)
 - 10.2 [AppSidebar](#)
 - 10.3 [AppHeader](#)
11. [Page-by-Page Breakdown](#)
 - 11.1 [Login & Registration](#)
 - 11.2 [Create Store](#)
 - 11.3 [Dashboard](#)
 - 11.4 [Products](#)
 - 11.5 [Orders](#)
 - 11.6 [Customers](#)
 - 11.7 [Categories](#)
 - 11.8 [Analytics](#)
 - 11.9 [Marketing \(Coupons\)](#)
 - 11.10 [Store Settings & Theme Customizer](#)
 - 11.11 [COD Reconciliation](#)

- 11.12 [Social Import](#)
 - 12. [Internationalization \(i18n\)](#)
 - 13. [Theme Engine — Storefront Customization](#)
 - 14. [Error Tracking & Observability](#)
 - 15. [CI/CD Pipelines](#)
 - 16. [Security Considerations](#)
 - 17. [Environment Configuration](#)
 - 18. [Local Development Setup](#)
 - 19. [Architecture Decisions & Trade-offs](#)
 - 20. [What I Would Change — Senior Perspective](#)
-

1. Executive Summary

NUMU Merchant Hub is a production-grade, single-page merchant dashboard built with **React 18 + TypeScript**. It is the control center for NUMU's e-commerce platform — the application through which merchants manage their online stores, products, orders, customers, analytics, marketing campaigns, and storefront themes.

The application is a **multi-tenant, multi-store** system: a single authenticated user can own and switch between multiple stores. Each store gets its own subdomain on the NUMU storefront (e.g., `my-brand.numu.io`). The dashboard communicates exclusively with the **NUMU API** (a FastAPI Python backend) via a custom `fetch`-based HTTP client with automatic JWT token refresh.

Key Characteristics

Dimension	Implementation
Framework	React 18.3 with TypeScript 5.6
Build tool	Vite 5 with SWC (sub-second HMR)
Styling	Tailwind CSS 3 + shadcn/ui (Radix primitives)
Server state	TanStack React Query v5
Client state	React Context (3 providers)
Forms	react-hook-form + Zod validation
Routing	react-router-dom v6 with nested layouts
i18n	i18next — English + Arabic (RTL)
Charts	Recharts (area, bar, pie/donut)
Error tracking	Sentry with browser tracing + session replay
CI/CD	GitHub Actions (lint → test → build → Sentry release)
API target	NUMU FastAPI backend at <code>localhost:8021</code>
Dev server port	8080

The codebase is clean, well-organized, and follows modern React patterns. It is a mature merchant dashboard that would be right at home competing with Shopify's admin panel or Salla's merchant interface.

2. Technology Stack

Core Runtime

Package	Version	Purpose
react	18.3.1	UI rendering library
react-dom	18.3.1	DOM renderer
typescript	5.6.2	Type safety
vite	5.4.14	Build tool + dev server
@vitejs/plugin-react-swc	3.5.0	SWC-based JSX transform (faster than Babel)

UI & Styling

Package	Version	Purpose
tailwindcss	3.4.17	Utility-first CSS framework
@radix-ui/*	Various	Headless accessible UI primitives (30+ packages)
class-variance-authority	0.7.1	Variant-based component styling
clsx + tailwind-merge	—	Conditional class composition
lucide-react	0.462.0	Icon library (500+ icons)
cmdk	1.0.4	Command palette (accessible combobox)

Data & State

Package	Version	Purpose
@tanstack/react-query	5.56.2	Server state management, caching, background refetch
react-hook-form	7.54.2	Performant form state management
@hookform/resolvers	3.9.1	Zod schema integration for forms
zod	3.23.8	Runtime schema validation

Routing & Navigation

Package	Version	Purpose
react-router-dom	6.26.2	Client-side routing with nested layouts

Internationalization

Package	Version	Purpose
i18next	24.2.2	i18n framework

react-i18next	15.4.1	React bindings for i18next
i18next-browser-languagedetector	8.0.4	Auto-detect browser language

Charting & Visualization

Package	Version	Purpose
recharts	2.15.0	Declarative chart components

Utilities

Package	Version	Purpose
date-fns	3.6.0	Date formatting and manipulation
sonner	1.7.4	Toast notification system
input-otp	1.4.1	OTP input component
react-day-picker	8.10.1	Calendar/date picker
embla-carousel-react	8.5.1	Carousel component
react-resizable-panels	2.1.7	Resizable panel layout
vaul	1.1.2	Mobile-friendly drawer component
next-themes	0.4.4	Dark/light mode management

Observability

Package	Version	Purpose
@sentry/react	9.15.0	Error tracking, performance monitoring

Dev Tools

Package	Version	Purpose
vitest	2.1.9	Unit testing framework
@testing-library/react	16.1.0	DOM testing utilities
jsdom	25.0.1	Browser DOM simulation for tests
eslint	9.9.1	Linting
openapi-typescript	7.6.1	Generate TS types from backend OpenAPI schema
postcss + autoprefixer	Various	CSS processing pipeline

3. Project Structure & Architecture

```

numo-merchant-hub/
├── .env                                # Environment variables (API URL, storefront URL)
├── .github/workflows/                 # CI/CD pipelines
│   ├── ci.yml                        # Lint → Test → Build pipeline
│   ├── security.yml                  # npm audit + dependency review
│   └── sentry-release.yml             # Sentry source map upload
├── components.json                    # shadcn/ui CLI configuration
├── index.html                         # SPA entry point (Vite HTML template)
├── package.json                       # Dependencies & scripts
├── tailwind.config.ts                 # Tailwind theme tokens & custom config
├── tsconfig.json                      # TypeScript base config (path aliases)
├── tsconfig.app.json                  # App-specific TS config
├── vite.config.ts                     # Vite bundler config (port 8080, SWC, aliases)
├── vitest.config.ts                   # Test runner config
├── public/
│   └── robots.txt
└── src/
    ├── main.tsx                      # App entry – Sentry init, ReactDOM.createRoot
    ├── App.tsx                       # Root component – providers, router, routes
    ├── App.css                       # Global styles
    ├── index.css                     # Tailwind directives + CSS custom properties
    ├── vite-env.d.ts                 # Vite type declarations
    ├── components/
    │   ├── layout/                   # DashboardLayout, AppSidebar, AppHeader
    │   ├── ui/                       # 48 shadcn/ui components
    │   ├── NumuLoader/               # Full-screen loading animation
    │   └── theme-editor/              # Storefront theme customization components
    ├── contexts/                     # React Context providers
    │   ├── AuthContext.tsx           # JWT auth, login/register/logout
    │   ├── StoreContext.tsx          # Multi-store management
    │   └── LanguageContext.tsx       # i18n language/direction switching
    ├── data/                         # Static/mock data
    │   ├── mock-social.ts           # Mock social media data for SocialImport
    │   └── theme-schemas.ts          # Bundled theme settings schema definitions
    ├── hooks/                        # Custom React hooks
    │   └── useThemeSettings.ts        # React Query hooks for theme CRUD
    ├── i18n/                         # Translation files
    │   ├── index.ts                  # i18next initialization & config
    │   ├── en.ts                     # English translations
    │   └── ar.ts                     # Arabic translations
    ├── lib/                          # Utility functions
    │   ├── utils.ts                  # cn() helper (clsx + tailwind-merge)
    │   └── storefront.ts             # Storefront URL builder
    ├── pages/                        # Route-level page components
    │   ├── Login.tsx
    │   ├── CreateStore.tsx
    │   ├── Dashboard.tsx
    │   ├── Products.tsx
    │   ├── Orders.tsx
    │   ├── Customers.tsx
    │   └── Categories.tsx

```

```

|   |─ Analytics.tsx
|   |─ Marketing.tsx
|   |─ StoreSettings.tsx
|   |─ CODReconciliation.tsx
|   |─ SocialImport.tsx
|   └─ NotFound.tsx
|─ services/                # API service layer
|   |─ api.ts               # Base HTTP client with token refresh
|   |─ authApi.ts          # Authentication endpoints
|   |─ storeApi.ts         # Store CRUD + shipping settings
|   |─ productApi.ts       # Product CRUD + adapter pattern
|   |─ orderApi.ts         # Order management + bulk operations
|   |─ customerApi.ts      # Customer listing
|   |─ analyticsApi.ts     # Dashboard & analytics data
|   |─ categoryApi.ts      # Category CRUD
|   |─ couponApi.ts        # Coupon/discount management
|   └─ themeApi.ts         # Theme customization + V2 section engine
└─ types/                  # Shared TypeScript types
    └─ openapi.d.ts        # Auto-generated from backend OpenAPI schema

```

Architectural Pattern

The application follows a **layered architecture**:

Pages (Route Components)	
Dashboard, Products, Orders, Analytics, etc.	
Layout Components	
DashboardLayout, AppSidebar, AppHeader	
UI Components (shadcn/ui)	
48 Radix-based primitives	
React Query Hooks	
useQuery, useMutation wrappers	
Services Layer	
apiClient → productApi, orderApi, themeApi...	
Context Providers	
AuthContext, StoreContext, LanguageContext	
Base Infrastructure	
Vite, TypeScript, Tailwind, Sentry	

Data flows **downward** through providers and **outward** through the service layer. Pages compose UI components, call React Query hooks (or call services directly), and use context for cross-cutting concerns (auth, current store, language).

4. Build System & Developer Experience

Vite Configuration

```
// vite.config.ts (simplified)
export default defineConfig({
  server: {
    host: "::",
    port: 8080,
  },
  plugins: [react()], // @vitejs/plugin-react-swc
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
  },
});
```

Key decisions:

- **Port 8080** — avoids conflict with the backend (8021) and storefront (8081)
- **SWC plugin** — uses Rust-based SWC instead of Babel for JSX transformation; ~20x faster HMR
- **Path alias @/** — maps to ./src/ , eliminating relative import hell (../../../../components/...)
- **IPv6 host ::** — listens on all interfaces (enables network access for mobile testing)

NPM Scripts

Script	Command	Purpose
dev	vite	Start dev server on port 8080
build	tsc && vite build	Type-check then produce optimized bundle
preview	vite preview	Serve production build locally
lint	eslint .	Run ESLint across all files
test	vitest run	Run test suite once
test:watch	vitest	Run tests in watch mode
generate:types	npx openapi-typescript http://localhost:8021/api/v1/openapi.json -o src/types/openapi.d.ts	Generate TypeScript definitions from backend OpenAPI schema

The `generate:types` script is noteworthy — it connects directly to the running backend to fetch its OpenAPI spec and generates TypeScript types. This is the **contract bridge** between the Python API and the TypeScript frontend.

Tailwind Configuration

The Tailwind config extends the default theme with:

- **CSS custom properties** for all colors (enables runtime dark mode switching)

- **Sidebar-specific colors** — 9 sidebar tokens (background, foreground, primary, accent, border, ring, plus their sub-variants)
- **Custom animations**: `accordion-down`, `accordion-up` for Radix accordion
- **Dark mode via class strategy** — toggled via a button in the header, persisted to `localStorage`

5. Application Entry Point & Initialization

main.tsx — Bootstrap Sequence

```
import * as Sentry from "@sentry/react";

// 1. Initialize Sentry BEFORE React renders
Sentry.init({
  dsn: import.meta.env.VITE_SENTRY_DSN,
  integrations: [
    Sentry.browserTracingIntegration(),
    Sentry.replayIntegration(),
  ],
  tracesSampleRate: 1.0, // 100% performance tracing
  replaysSessionSampleRate: 0.1, // 10% of sessions get full replay
  replaysOnErrorSampleRate: 1.0, // 100% replay on errors
});

// 2. Initialize i18n (side-effect import)
import "./i18n";

// 3. Render React app
createRoot(document.getElementById("root")!).render(<App />);
```

Initialization order matters:

1. Sentry hooks into `window.onerror` and `fetch` before any app code runs
2. i18n initializes with browser language detection and `localStorage` cache
3. React mounts the component tree

App.tsx — Provider Stack

QueryClientProvider	← Server state cache
└─ TooltipProvider	← Global tooltip context
└─ LanguageProvider	← i18n direction management
└─ AuthProvider	← JWT token lifecycle
└─ StoreProvider	← Multi-store selection
└─ BrowserRouter	← Client-side routing
└─ Routes	← Page routing
└─ Toaster	← Toast notification outlet

The **QueryClient** is configured with:

- `staleTime: 5 * 60 * 1000` (5 minutes) — data considered fresh for 5 min, no refetch
- `gcTime: 10 * 60 * 1000` (10 minutes) — unused cache entries garbage collected after 10 min

This is a sensible default for a dashboard where data doesn't change every second but should stay relatively fresh.

6. Routing & Navigation Guards

Route Table

```
<Routes>
  { /* Public route */}
  <Route path="/login" element={<Login />} />

  { /* Authenticated but no store required */}
  <Route element={<RequireAuth />}>
    <Route path="/create-store" element={<CreateStore />} />
  </Route>

  { /* Authenticated + store required */}
  <Route element={<RequireAuth />}>
    <Route element={<RequireStore />}>
      <Route element={<DashboardLayout />}>
        <Route path="/" element={<Dashboard />} />
        <Route path="/products" element={<Products />} />
        <Route path="/orders" element={<Orders />} />
        <Route path="/store" element={<StoreSettings />} />
        <Route path="/cod" element={<CODReconciliation />} />
        <Route path="/social" element={<SocialImport />} />
        <Route path="/customers" element={<Customers />} />
        <Route path="/analytics" element={<Analytics />} />
        <Route path="/marketing" element={<Marketing />} />
        <Route path="/categories" element={<Categories />} />
      </Route>
    </Route>
  </Route>

  <Route path="*" element={<NotFound />} />
</Routes>
```

Guard Components

RequireAuth — Renders `<Outlet />` if `isAuthenticated` is true. Otherwise redirects to `/login`. Shows `<NumLoadingScreen />` while auth state is being validated (initial `GET /auth/me` call).

RequireStore — After auth succeeds, checks if the user has at least one store. If `hasStores` is false, redirects to `/create-store`. This ensures a merchant always has a store context before entering the dashboard.

Navigation Flow

```
User visits /products
└─ RequireAuth checks token
   └─ No token → redirect to /login
   └─ Has token → validates via GET /auth/me
```

```
└─ Invalid → clears tokens, redirect to /login
└─ Valid → RequireStore checks stores
    └─ No stores → redirect to /create-store
    └─ Has stores → renders DashboardLayout → Products
```

7. State Management — Context Providers

The application uses **three React Context providers** for global client-side state. This is a deliberate choice over a state management library like Redux or Zustand — the state surface is small enough (auth + current store + language) that Context is the right tool.

7.1 AuthContext — JWT Authentication

File: `src/contexts/AuthContext.tsx`

Token Storage

Key	Storage	Content
numu-token	localStorage	JWT access token (short-lived)
numu-refresh-token	localStorage	JWT refresh token (long-lived)

State Shape

```
interface AuthContextType {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  register: (data: RegisterData) => Promise<void>;
  logout: () => void;
}
```

Token Handoff from Landing Page

A critical feature: the landing page (a separate app at `numu.io`) handles initial registration and generates tokens. It redirects to the merchant hub with tokens in URL parameters:

```
/login?token=eyJ...&refresh_token=eyJ...
```

AuthContext checks for these URL parameters on mount. If found, it stores the tokens, clears the URL (using `window.history.replaceState`), and validates via `GET /auth/me`. This is a **cross-origin auth handoff pattern** that avoids cookies and CORS complexity.

Validation Flow

Mount:

1. Check URL params for token + refresh_token
→ If found: store in localStorage, clear URL
2. Read tokens from localStorage
3. Call `GET /auth/me` with access token

```
└─ 200: Set user, isAuthenticated = true
└─ 401: Token expired
    └─ Try refresh via POST /auth/refresh-token
        └─ 200: Store new tokens, retry GET /auth/me
            └─ Fail: Clear tokens, set isAuthenticated = false
└─ Redirect to /login
```

7.2 StoreContext — Multi-Store Management

File: `src/contexts/StoreContext.tsx`

After authentication, StoreContext fetches the user's stores via `GET /stores/`. It manages which store is currently selected and persists the selection in `localStorage` under key `numu-current-store`.

```
interface StoreContextType {
  stores: Store[];
  currentStore: Store | null;
  setCurrentStore: (store: Store) => void;
  switchStore: (storeId: string) => void;
  isLoading: boolean;
  hasStores: boolean;
  refetchStores: () => void;
}
```

Multi-store switching is available from the `AppHeader` via a dropdown. When the merchant switches stores, StoreContext updates the current store and all React Query hooks that use `storeId` in their query keys automatically invalidate and refetch.

7.3 LanguageContext — i18n & RTL

File: `src/contexts/LanguageContext.tsx`

Manages the active language and toggles the HTML `dir` attribute between `ltr` and `rtl`.

```
interface LanguageContextType {
  language: "en" | "ar";
  setLanguage: (lang: "en" | "ar") => void;
  toggleLanguage: () => void;
}
```

When toggled:

1. Calls `i18n.changeLanguage(newLang)`
2. Sets `document.dir` to `rtl` or `ltr`
3. Sets `document.lang` to `ar` or `en`
4. Tailwind's RTL utilities (`rtl:` , `ltr:`) activate automatically

The entire UI, including the sidebar, flips direction when switching to Arabic. This is **true bi-directional** support, not just text translation.

8. API Layer — Service Architecture

8.1 Base API Client

File: `src/services/api.ts`

The foundation of all backend communication is a custom `apiClient<T>()` function built on the native `fetch` API. No Axios — this is a deliberate choice to keep the bundle lean and avoid an 11KB dependency for something `fetch` handles natively.

```
async function apiClient<T>(  
  endpoint: string,  
  options?: RequestInit & { rawBody?: boolean }  
) : Promise<T>
```

Features

1. **Automatic Bearer token injection** — reads `numu-token` from `localStorage` and adds `Authorization: Bearer <token>` header
2. **Content-Type handling** — auto-sets `application/json` unless the body is `FormData` (in which case the browser sets the multipart boundary)
3. **204 No Content** — returns `null` as `T` for DELETE responses
4. **401 Auto-Refresh with deduplication** — the most sophisticated part:

```
Request → 401 Unauthorized  
└─ Is a refresh already in progress?  
  └─ Yes: await the shared refreshPromise  
  └─ No: Start refresh, store promise in module-level variable  
      └─ POST /auth/refresh-token  
          └─ 200: Store new tokens, retry original request  
          └─ Fail: Clear tokens, redirect to /login
```

The key insight is the **shared promise pattern**: if multiple requests fail with 401 simultaneously, they all await the same refresh call instead of firing N refresh requests. This prevents token refresh race conditions — a common bug in SPA authentication.

8.2 Auth Service

File: `src/services/authApi.ts`

Function	Method	Endpoint	Purpose
<code>login()</code>	POST	<code>/auth/login</code>	Email + password login
<code>register()</code>	POST	<code>/auth/register</code>	New merchant registration
<code>getMe()</code>	GET	<code>/auth/me</code>	Validate token, get current user
<code>refreshToken()</code>	POST	<code>/auth/refresh-token</code>	Exchange refresh token for new pair

RegisterData includes: `email`, `password`, `first_name`, `last_name`, `phone` (optional).

8.3 Store Service

File: `src/services/storeApi.ts`

Function	Method	Endpoint	Purpose
listStores()	GET	/stores/	All stores for current user
createStore()	POST	/stores/	Create new store
getStore()	GET	/stores/{id}	Single store details
updateStore()	PUT	/stores/{id}	Update store settings
checkSubdomain()	GET	/stores/check-subdomain/{slug}	Subdomain availability
getShippingSettings()	GET	/stores/{id}/shipping	Shipping zone config
updateShippingSettings()	PUT	/stores/{id}/shipping	Update shipping config
getShippingCarriers()	GET	/stores/{id}/shipping/carriers	Available carriers
updateShippingCarrier()	PUT	/stores/{id}/shipping/carriers/{cid}	Update carrier config

Supported shipping carriers: Aramex, Bosta, Mylerz, Manual (custom).

8.4 Product Service & Adapter Pattern

File: `src/services/productApi.ts`

This is architecturally the most interesting service because it implements a **data adapter layer** between the backend's data model and the frontend's domain model.

The Problem

The backend API returns products in a format optimized for storage:

```
{
  "status": "active",
  "attributes": { "name_ar": "قميص", "description_ar": "..."},
  "compare_at_price": 199.99
}
```

But the frontend needs:

```
{
  status: "published",           // status mapping
  nameAr: "قميص",              // flattened from attributes
  descriptionAr: "...",         // flattened from attributes
  compareAtPrice: 199.99        // camelCase
}
```

The Solution — Adapter Functions

Three adapter functions bridge this gap:

1. `apiToProduct(api)` — Transforms API response → frontend `Product` type

- Maps `"active"` → `"published"`, `"inactive"` → `"draft"`

- Extracts `name_ar` and `description_ar` from nested `attributes` object
 - Converts `snake_case` to `camelCase`
 - Normalizes image URLs
2. `productToApiCreate(form)` — Transforms frontend form data → API create payload
- Reverses status mapping: `"published"` → `"active"`
 - Nests Arabic fields back into `attributes`
 - Converts `camelCase` to `snake_case`
3. `productToApiUpdate(form)` — Same as create but for PATCH semantics (partial updates)

Why this matters: This is a **hexagonal architecture boundary** — the frontend defines its own domain model and never leaks backend implementation details into UI components. If the API changes how it stores Arabic names, only the adapter functions change, not 15 different components.

Product CRUD Operations

Function	Method	Endpoint	Purpose
<code>listProducts()</code>	GET	<code>/stores/{id}/products</code>	Paginated list with filters
<code>getProduct()</code>	GET	<code>/stores/{id}/products/{pid}</code>	Single product
<code>createProduct()</code>	POST	<code>/stores/{id}/products</code>	Create product
<code>updateProduct()</code>	PUT	<code>/stores/{id}/products/{pid}</code>	Full update
<code>deleteProduct()</code>	DELETE	<code>/stores/{id}/products/{pid}</code>	Delete product
<code>uploadProductImage()</code>	POST	<code>/stores/{id}/products/{pid}/images</code>	Upload image (FormData)
<code>deleteProductImage()</code>	DELETE	<code>/stores/{id}/products/{pid}/images/{iid}</code>	Remove image

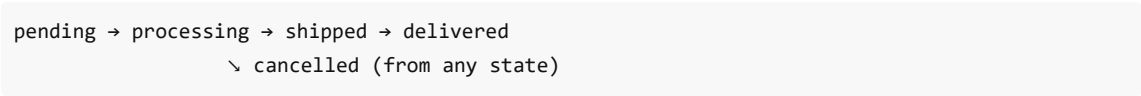
8.5 Order Service

File: `src/services/orderApi.ts`

Orders are the commercial heart of the platform.

Function	Method	Endpoint	Purpose
<code>listOrders()</code>	GET	<code>/stores/{id}/orders</code>	List with filters (status, payment, fulfillment, dates, search)
<code>getOrder()</code>	GET	<code>/stores/{id}/orders/{oid}</code>	Order detail with line items
<code>updateOrderStatus()</code>	PUT	<code>/stores/{id}/orders/{oid}/status</code>	Change order status
<code>cancelOrder()</code>	POST	<code>/stores/{id}/orders/{oid}/cancel</code>	Cancel order
<code>getOrderTimeline()</code>	GET	<code>/stores/{id}/orders/{oid}/timeline</code>	Order history/audit trail
<code>bulkUpdateStatus()</code>	POST	<code>/stores/{id}/orders/bulk-status</code>	Batch status change
<code>markOrderPaid()</code>	POST	<code>/stores/{id}/orders/{oid}/mark-paid</code>	Manual payment confirmation

Order Status Workflow:



The frontend enforces **sequential status transitions** — you cannot jump from `pending` to `delivered` without going through `processing` and `shipped` .

Type Definitions

```
interface Order {
  id: string;
  order_number: string;
  status: "pending" | "processing" | "shipped" | "delivered" | "cancelled";
  payment_status: "pending" | "paid" | "failed" | "refunded";
  fulfillment_status: "unfulfilled" | "partially_fulfilled" | "fulfilled";
  total: number;
  subtotal: number;
  shipping_cost: number;
  discount_amount: number;
  items: OrderLineItem[];
  shipping_address: OrderAddress;
  customer: { id: string; name: string; email: string; phone: string };
  payment_method: string;
  notes: string;
  created_at: string;
  updated_at: string;
}
```

8.6 Customer Service

File: `src/services/customerApi.ts`

A lightweight read-only service:

Function	Method	Endpoint	Purpose
<code>listCustomers()</code>	GET	<code>/stores/{id}/customers</code>	Paginated list with search
<code>getCustomer()</code>	GET	<code>/stores/{id}/customers/{cid}</code>	Customer detail

8.7 Analytics Service

File: `src/services/analyticsApi.ts`

Two groups of endpoints — dashboard-level and deep analytics.

Dashboard:

Function	Method	Endpoint	Purpose
<code>getDashboardStats()</code>	GET	<code>/stores/{id}/dashboard/stats</code>	KPI cards (revenue, orders, customers, AOV)

getRevenueChart()	GET	/stores/{id}/dashboard/revenue-chart	Revenue time series
getTopProducts()	GET	/stores/{id}/dashboard/top-products	Best sellers list

Analytics:

Function	Method	Endpoint	Purpose
getSalesOverview()	GET	/stores/{id}/analytics/sales-overview	Aggregated sales metrics
getSalesChart()	GET	/stores/{id}/analytics/sales-chart	Sales time series
getAnalyticsTopProducts()	GET	/stores/{id}/analytics/top-products	Extended top products
getSalesByLocation()	GET	/stores/{id}/analytics/sales-by-location	Geographic breakdown
getCustomerAnalytics()	GET	/stores/{id}/analytics/customers	Customer segments & metrics
getConversionStats()	GET	/stores/{id}/analytics/conversion	Funnel conversion data

All analytics endpoints accept a `period` parameter: `7d` , `30d` , or `90d` .

8.8 Category Service

File: `src/services/categoryApi.ts`

Function	Method	Endpoint	Purpose
listCategories()	GET	/stores/{id}/categories	All categories (flat list)
createCategory()	POST	/stores/{id}/categories	Create category
updateCategory()	PUT	/stores/{id}/categories/{cid}	Update category
deleteCategory()	DELETE	/stores/{id}/categories/{cid}	Delete category

Categories support `parent_id` for hierarchical nesting, `position` for manual ordering, and `is_active` for visibility control.

8.9 Coupon Service

File: `src/services/couponApi.ts`

Function	Method	Endpoint	Purpose
listCoupons()	GET	/stores/{id}/coupons	All coupons
getCoupon()	GET	/stores/{id}/coupons/{cid}	Coupon detail
createCoupon()	POST	/stores/{id}/coupons	Create coupon

updateCoupon()	PUT	/stores/{id}/coupons/{cid}	Update coupon
deleteCoupon()	DELETE	/stores/{id}/coupons/{cid}	Delete coupon

Coupon Types:

- `percentage` — Discount by percentage (e.g., 20% off)
- `fixed` — Fixed amount discount (e.g., 50 EGP off)
- `free_shipping` — Waive shipping cost

Targeting:

- `applies_to: "all" | "specific_products" | "specific_categories"`
- `product_ids: string[]` — When targeting specific products
- `category_ids: string[]` — When targeting specific categories

Constraints:

- `min_order_amount` — Minimum order value to apply
- `max_discount_amount` — Cap on percentage discounts
- `usage_limit` — Total number of uses allowed
- `valid_from / valid_until` — Date range validity

8.10 Theme Service & V2 Section Engine

File: `src/services/themeApi.ts`

The theme service manages storefront customization — how the merchant's public-facing shop looks.

Available Themes

Five built-in themes:

1. **default** — Clean, minimal modern design
2. **skeuomorphic** — Rich textures and depth
3. **neo-brutalism** — Bold, high-contrast, raw
4. **editorial** — Magazine-style layout
5. **luxury-minimal** — Refined, spacious, premium feel

V2 Section Engine

The most sophisticated API integration in the project. The V2 engine treats a storefront as a **list of configurable sections**:

```
interface SectionInstanceData {
  id: string;           // Unique instance ID
  type: string;         // Section type (e.g., "hero-banner", "featured-products")
  settings: Record<string, unknown>; // Section-specific settings
  disabled?: boolean;
}

interface TemplateConfigData {
  order: string[];      // Section rendering order
  sections: Record<string, SectionInstanceData>; // Section instances by ID
}
```

This is conceptually identical to **Shopify's section architecture** — merchants can:

- Add/remove sections to their storefront
- Reorder sections via drag-and-drop
- Configure each section's settings (images, text, colors, layout)
- Preview changes before publishing

Theme Schema System:

Each theme declares its settings schema:

```
interface SettingDefinition {
  key: string;
  type: "color" | "checkbox" | "image" | "text" | "select" | "font" | "range";
  label: string;
  labelAr: string; // Arabic label for RTL
  default: string | number | boolean;
  group?: string;
  groupAr?: string;
  options?: { label: string; labelAr: string; value: string }[];
  min?: number; max?: number; step?: number; unit?: string;
}
```

The dashboard renders a **dynamic settings form** based on these schemas — a form that auto-generates itself from the theme's schema definition. This means adding a new theme setting to the storefront automatically creates the corresponding UI control in the merchant dashboard.

Function	Method	Endpoint	Purpose
fetchThemes()	GET	/themes	Available themes list
fetchCustomization()	GET	/stores/{id}/customization	Current store's theme config
updateCustomization()	PUT	/stores/{id}/customization	Save draft changes
publishCustomization()	POST	/stores/{id}/customization/publish	Make changes live
fetchThemeSchemas()	GET	/themes/schemas	All theme settings schemas

9. UI Component Library

The project uses **shadcn/ui** — not a traditional npm package, but a **copy-paste component system** built on Radix UI primitives. Components live in `src/components/ui/` and are fully owned by the project.

48 UI Components

Component	Radix Primitive	Purpose
accordion	@radix-ui/react-accordion	Collapsible content sections
alert-dialog	@radix-ui/react-alert-dialog	Confirmation dialogs
alert	—	Status/warning messages

aspect-ratio	@radix-ui/react-aspect-ratio	Responsive media containers
avatar	@radix-ui/react-avatar	User/merchant avatars
badge	—	Status badges, tags
breadcrumb	—	Navigation breadcrumbs
button	—	Primary action element (with CVA variants)
calendar	react-day-picker	Date selection
card	—	Content containers
carousel	embla-carousel-react	Image/content carousels
chart	recharts	Chart wrapper with theming
checkbox	@radix-ui/react-checkbox	Boolean inputs
collapsible	@radix-ui/react-collapsible	Expandable sections
command	cmdk	Command palette / searchable select
context-menu	@radix-ui/react-context-menu	Right-click menus
dialog	@radix-ui/react-dialog	Modal windows
drawer	vaul	Mobile-friendly bottom sheets
dropdown-menu	@radix-ui/react-dropdown-menu	Action menus
form	react-hook-form	Form state + validation wrapper
hover-card	@radix-ui/react-hover-card	Hover-triggered content
input-otp	input-otp	One-time password input
input	—	Text input field
label	@radix-ui/react-label	Form labels
menubar	@radix-ui/react-menubar	Menu bar (app-level menus)
navigation-menu	@radix-ui/react-navigation-menu	Site navigation
pagination	—	Page number navigation
popover	@radix-ui/react-popover	Floating content
progress	@radix-ui/react-progress	Progress bars
radio-group	@radix-ui/react-radio-group	Radio button groups
resizable	react-resizable-panels	Draggable panel layouts
scroll-area	@radix-ui/react-scroll-area	Custom scrollbars
select	@radix-ui/react-select	Dropdown select

separator	@radix-ui/react-separator	Visual dividers
sheet	@radix-ui/react-dialog	Slide-in panels
sidebar	—	Dashboard sidebar
skeleton	—	Loading placeholder animations
slider	@radix-ui/react-slider	Range inputs
sonner	sonner	Toast notifications
switch	@radix-ui/react-switch	Toggle switches
table	—	Data tables
tabs	@radix-ui/react-tabs	Tab navigation
textarea	—	Multi-line text input
toast / toaster	—	Toast notification system
toggle / toggle-group	@radix-ui/react-toggle	Toggle buttons
tooltip	@radix-ui/react-tooltip	Hover tooltips

All components are:

- Fully accessible (WAI-ARIA compliant via Radix)
- Dark mode compatible (via CSS custom properties)
- RTL-aware (via Tailwind's `rtl` variant)
- Keyboard navigable
- Styled with CVA (class-variance-authority) for variant composition

10. Layout System

10.1 DashboardLayout

File: `src/components/layout/DashboardLayout.tsx`

Wraps all dashboard pages. Structure:

```

SidebarProvider
├─ AppSidebar (left/right depending on RTL)
└─ SidebarInset
  ├─ AppHeader (sticky top)
  ├─ Alerts (trial expiry, suspended, pending approval)
  └─ <main>
    └─ <Outlet /> (current page)
```

Alert System:

The layout renders contextual banners based on store state:

Condition	Banner	Color
-----------	--------	-------

Trial active, days > 0	"Demo Trial — X days remaining"	Yellow/Warning
Trial expired	"Trial Expired"	Red/Destructive
Store suspended	"Store Suspended"	Red/Destructive
Store pending approval	"Pending Approval"	Blue/Info

Trial days are calculated from `store.trial_ends_at` using `date-fns/differenceInDays` .

10.2 AppSidebar

File: `src/components/layout/AppSidebar.tsx`

A collapsible sidebar with three navigation groups:

Main:

- Dashboard (`/`)
- Orders (`/orders`)
- Products (`/products`)
- Categories (`/categories`)
- Customers (`/customers`)

Channels:

- Marketing (`/marketing`)
- Online Store (`/store`)
- Social Import (`/social`)

Insights:

- Analytics (`/analytics`)
- COD Reconciliation (`/cod`)

Features:

- Collapsible groups with chevron indicators
- Active route highlighting (compares `location.pathname`)
- NUMU logo + branding at top
- Settings gear icon in footer
- Sidebar flips position in RTL mode (right-side for Arabic)
- Responsive: collapses via `SidebarTrigger` button

10.3 AppHeader

File: `src/components/layout/AppHeader.tsx`

The top header bar contains:

Element	Position	Functionality
Sidebar trigger	Far left	Toggle sidebar collapse
Breadcrumb separator	Left	Visual anchor
Search input	Center-left	Placeholder search (products, orders)

Date display	Center	Today's date in localized format
Dark mode toggle	Right	Sun/Moon icon toggle, persisted to <code>localStorage</code>
Language switcher	Right	"EN"/"AR" button, toggles direction
Store selector	Right	Dropdown with all stores + "Create new store"
Notification bell	Right	Notification icon (UI-only currently)
Profile avatar	Far right	Dropdown with user info + logout

Dark Mode Implementation:

```
const toggleDarkMode = () => {
  const newMode = !isDarkMode;
  setIsDarkMode(newMode);
  document.documentElement.classList.toggle("dark", newMode);
  localStorage.setItem("numu-dark-mode", String(newMode));
};
```

Simple, effective, no external library for this — `next-themes` is installed but the header uses its own implementation for direct control.

11. Page-by-Page Breakdown

11.1 Login & Registration

File: `src/pages/Login.tsx`

Layout: Split-screen design:

- **Left half** (desktop only): Decorative gradient background with NUMU branding, tagline, and a floating illustration
- **Right half:** Login/register form

Features:

- Toggle between **Login** and **Register** modes via tab
- Login: email + password
- Register: first name, last name, email, password
- Form validation (basic, not Zod — direct state management)
- Error display via inline messages
- Loading spinner during API calls
- On success: stores tokens, navigates to `/` (or `/create-store` if no stores)
- NUMU logo shown on mobile (replaces decorative left panel)

11.2 Create Store

File: `src/pages/CreateStore.tsx`

Purpose: First-time store setup for new merchants.

Features:

- Store name input
- **Auto-generated subdomain** from store name (slugified)
- **Real-time subdomain availability check** — calls `GET /stores/check-subdomain/{slug}` with debounce
- Domain suffix display (e.g., `.numu.io` or `.localhost:8081` in dev)
- Language selector (default store language)
- Currency selector
- Same split-screen decorative layout as Login
- On success: redirects to dashboard `/`

11.3 Dashboard

File: `src/pages/Dashboard.tsx`

The main landing page after login. A data-rich overview of the store's performance.

Greeting Section:

- Time-of-day greeting: "Good morning/afternoon/evening" based on system clock
- Merchant's first name
- "Here's what's happening with your store today"

KPI Cards (4 cards, top row):

Card	Data	Animation
Today's Revenue	Formatted currency (EGP)	Count-up from 0
Today's Orders	Integer count	Count-up from 0
New Customers	Integer count	Count-up from 0
Avg Order Value	Formatted currency	Count-up from 0

Each card shows a change percentage vs. previous period with colored arrow (green up, red down).

Period Selector: Toggle between 7 days, 30 days, 90 days — updates all charts and metrics.

Revenue Trend Chart:

- Recharts `AreaChart` with gradient fill
- X-axis: dates, Y-axis: currency
- Responsive container, smooth curve type
- Tooltip with formatted values

Order Status Donut Chart:

- Recharts `PieChart` in donut configuration
- Segments: Pending, Processing, Shipped, Delivered
- Color-coded segments
- Center label shows total orders

Top Products Table:

- Rank, product name, units sold
- Top 5 products by sales volume

Recent Orders Table:

- Order number, customer name, total, status badge, date
- "View All" link to `/orders`

Data Loading: Uses React Query with `keepPreviousData` — the old data stays visible while new data for a different period is loading, preventing flickering.

11.4 Products

File: `src/pages/Products.tsx`

The product catalog management page.

Features:

1. **Search** — Debounced (300ms) search input, triggers server-side search
2. **Status filter tabs** — All / Published / Draft / Archived
3. **Category filter** — Dropdown filter by category
4. **Paginated table** — 20 products per page, server-side pagination
5. **Product table columns:**
 - Thumbnail image
 - Product name (+ Arabic name if available)
 - Price (formatted as EGP)
 - Compare-at price (strikethrough original price)
 - Stock quantity
 - Status badge (colored)
 - Actions dropdown (Edit, Delete)
6. **Add/Edit Product Dialog:**
 - Product name (English)
 - Product name (Arabic)
 - Description (English)
 - Description (Arabic)
 - Price
 - Compare-at price
 - Stock quantity
 - Status selector (Published / Draft / Archived)
 - Category selector
 - **Variants** — Dynamic variant builder (name + comma-separated options)
 - **Image upload** — Multi-image upload with preview, drag-and-drop zone
7. **Delete confirmation** — AlertDialog with "are you sure" prompt
8. **Toast notifications** — Success/error feedback via Sonner

Data Flow:

```
Products page
→ useQuery("products", listProducts) ← Server data
→ Adapter: apiToProduct() for display ← Transform
→ Table rendering ← UI
```


→ Edit form	← User input
→ Adapter: <code>productToApiUpdate()</code>	← Transform back
→ <code>useMutation(updateProduct)</code>	← Send to server
→ <code>invalidateQueries("products")</code>	← Auto-refresh list

11.5 Orders

File: `src/pages/Orders.tsx`

The order management page — the operational core of the dashboard.

Features:

1. **Status filter tabs** — All / Pending / Processing / Shipped / Delivered / Cancelled
2. **Extended filters** — Payment status, fulfillment status, date range, search
3. **Bulk selection** — Checkbox per row + "select all" for batch operations
4. **Bulk status update** — Change status of multiple orders at once
5. **Order table columns:**
 - Checkbox (for bulk selection)
 - Order number (#1001, etc.)
 - Customer name
 - Total amount
 - Order status (colored badge)
 - Payment status (colored badge)
 - Fulfillment status
 - Date
 - Actions
6. **Order Detail View** (slide-in sheet or full view):
 - Order header with order number and status
 - Customer information (name, email, phone)
 - Shipping address
 - Line items table (product, quantity, price, subtotal)
 - Order summary (subtotal, shipping, discounts, total)
 - Payment information
 - **Status workflow buttons** — Sequential status progression
 - **Mark as Paid** — Manual payment confirmation for COD
 - **Cancel order** — With confirmation dialog
7. **Order Timeline** — Chronological history of order events:
 - Order created
 - Payment received
 - Status changed to processing
 - Shipped via Aramex (tracking #)
 - Delivered
 - Each event shows timestamp and actor

Status Transition Enforcement:

```
// Only allows the next logical status
const getNextStatus = (current: string) => {
  switch (current) {
    case "pending": return "processing";
    case "processing": return "shipped";
    case "shipped": return "delivered";
    default: return null;
  }
};
```

11.6 Customers

File: `src/pages/Customers.tsx`

A read-only customer listing.

Features:

- Search by name, email, or phone
- Paginated table
- Columns: name, email, phone, orders count, total spent, join date
- Click to view customer detail (if implemented)

11.7 Categories

File: `src/pages/Categories.tsx`

Product category management.

Features:

- Category list with name, product count, position, status (active/inactive)
- Add/Edit category dialog
- Category fields: name, description, parent category (for nesting), position (sort order), is_active toggle
- Delete with confirmation
- Drag-and-drop reordering (via position field)

11.8 Analytics

File: `src/pages/Analytics.tsx`

Deep-dive analytics beyond the dashboard overview.

6 React Query Hooks (all with period parameter):

1. `getSalesOverview()` — Aggregate metrics: total revenue, total orders, average order value, refund rate
2. `getSalesChart()` — Time series for revenue and orders plotted together
3. `getAnalyticsTopProducts()` — Extended top products with revenue and units
4. `getSalesByLocation()` — Geographic breakdown of sales (by Egyptian governorate/city)
5. `getCustomerAnalytics()` — New vs returning customers, lifetime value, churn
6. `getConversionStats()` — Funnel: visits → add to cart → checkout → purchase

UI Layout:

- Period selector (7d / 30d / 90d) at top
- KPI summary cards row

- Revenue + orders dual-axis chart
- Top products bar chart
- Sales by location horizontal bar chart
- Conversion funnel visualization
- Customer segment breakdown

11.9 Marketing (Coupons)

File: `src/pages/Marketing.tsx`

Coupon and discount management.

Features:

- Coupon list with code, type, value, usage stats, status, validity dates
- Add/Edit coupon with full targeting:
 - Code (auto-generate or custom)
 - Type: Percentage / Fixed / Free Shipping
 - Value (percentage or fixed amount)
 - Minimum order amount
 - Maximum discount (cap for percentage coupons)
 - Usage limit
 - Valid from / Valid until
 - **Applies to:** All products, Specific products, Specific categories
 - Product/Category multi-select with search (fetches all products for targeting)
- Delete coupon with confirmation
- Status badge: Active / Expired / Upcoming

11.10 Store Settings & Theme Customizer

File: `src/pages/StoreSettings.tsx`

Three-tab settings page:

Tab 1 — General:

- Store name
- Store description
- Contact email
- Store logo upload
- Default language
- Currency

Tab 2 — Theme Customizer:

The most complex UI in the application. A **Shopify-style theme editor** that lets merchants customize their storefront.

Sub-features:

- **Theme selector** — Choose from 5 built-in themes (default, skeuomorphic, neo-brutalism, editorial, luxury-minimal)
- **Font picker** — Google Fonts integration with real-time preview, preloads selected font
- **Color settings** — Primary, secondary, accent, background, text colors via color picker
- **Layout settings** — Border radius (range slider), button style (rounded/square/pill)
- **V2 Section editor:**

- List of storefront sections in order
- Add new section from available section types
- Remove sections
- Reorder sections (drag-and-drop)
- Configure each section's individual settings
- Enable/disable sections without removing
- **Save draft** — Saves changes without publishing
- **Publish** — Makes changes live on the storefront

Theme Hook Integration:

```
const { data: customization } = useCustomization(); // Current theme config
const { mutate: save } = useUpdateCustomization(); // Save draft
const { mutate: publish } = usePublishCustomization(); // Go live
const { data: themes } = useAvailableThemes(); // Theme list
```

Tab 3 — Shipping:

- Shipping zone configuration
- Carrier integration settings (Aramex, Bosta, Mylerz)
- Manual shipping rate configuration
- Free shipping thresholds

11.11 COD Reconciliation

File: `src/pages/CODReconciliation.tsx`

Cash-on-Delivery reconciliation dashboard — critical for Egyptian e-commerce where COD is the dominant payment method.

Currently uses mock data (no API integration yet), but the UI is fully built:

- COD KPI cards: Total COD collected, Pending reconciliation, Reconciled amount, Outstanding balance
- Delivery partner breakdown
- Payment timeline
- Individual delivery reconciliation rows

Why this matters: In Egypt, ~70% of e-commerce transactions are COD. Merchants need to track money collected by delivery drivers and reconcile it against actual orders. This page is the financial control panel for that workflow.

11.12 Social Import

File: `src/pages/SocialImport.tsx`

Import products from social media platforms (Instagram, Facebook).

Features:

- Connect/disconnect social media accounts
- View social media posts with engagement metrics (likes, comments)
- Filter by platform (All / Instagram / Facebook)
- Select posts to import as products
- Bulk import selected posts
- Shows import status (imported/not imported)

- Currently uses **mock data** — API integration pending

Why this exists: Many Egyptian small businesses sell through Instagram and Facebook before setting up a proper e-commerce store. This feature lets them migrate their product catalog from social media posts directly into their NUMU store.

12. Internationalization (i18n)

Architecture

```
src/i18n/
├─ index.ts    ← i18next initialization
├─ en.ts       ← English translations (full namespace)
└─ ar.ts       ← Arabic translations (Egyptian dialect)
```

Configuration

```
i18n
.use(LanguageDetector)    // Auto-detect from localStorage → navigator
.use(initReactI18next)    // React integration
.init({
  resources: { en: { translation: en }, ar: { translation: ar } },
  fallbackLng: "en",
  interpolation: { escapeValue: false }, // React handles XSS
  detection: {
    order: ["localStorage", "navigator"],
    caches: ["localStorage"],
  },
});
```

Translation Namespaces

The translation files are organized by feature:

Namespace	Example Keys
nav.*	dashboard, products, orders, store, analytics, etc.
header.*	storeName, notifications, darkMode, search
dashboard.*	welcome, todayRevenue, revenueTrend, trialTitle, etc.
products.*	title, addProduct, search, variants, uploadImages
orders.*	Order status labels, actions, timeline events
customers.*	Customer table headers and labels
analytics.*	Chart labels, metric names
marketing.*	Coupon fields, targeting labels

categories.*	Category management labels
store.*	Store settings, theme labels
social.*	Social import labels

Arabic Dialect

The Arabic translations use **Egyptian Arabic** (العامية المصرية), not Modern Standard Arabic:

- "ابحث عن المنتجات" instead of "دور على منتجات"
- "لا توجد منتجات" instead of "مفیش منتجات"
- "تم بيعه" instead of "اتباع"

This is a deliberate product decision — NUMU targets the Egyptian market, and Egyptian Arabic feels more natural to Egyptian merchants.

RTL Support

When Arabic is active:

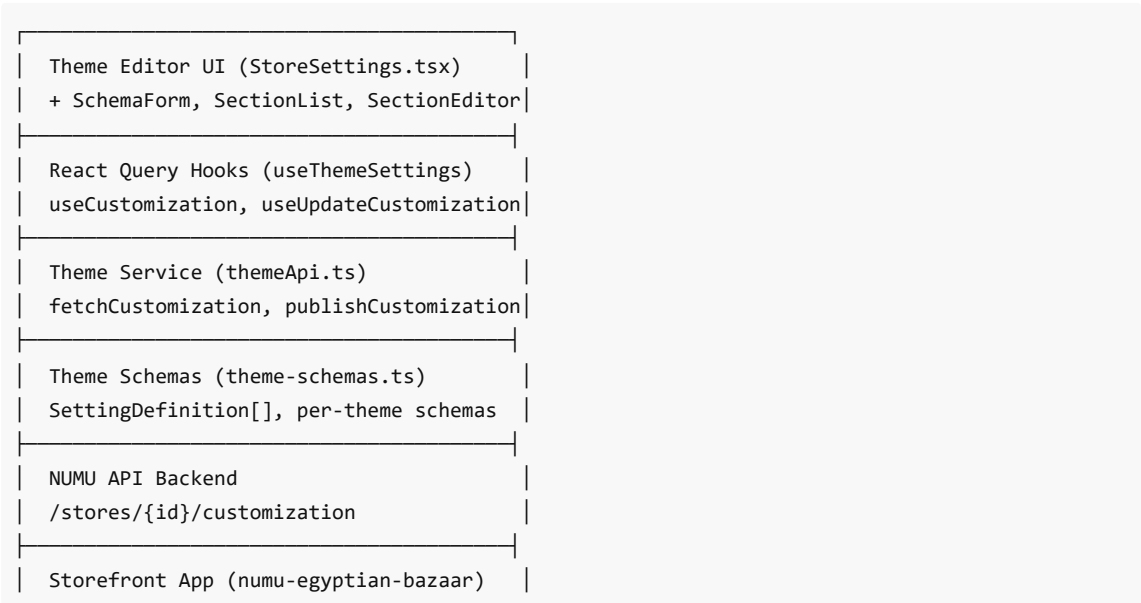
1. `document.dir = "rtl"` — Entire layout flips
2. `document.lang = "ar"` — Proper screen reader announcements
3. All Tailwind `rtl`: variants activate (sidebar moves to right side, text aligns right, etc.)
4. Date formatting switches to Arabic locale
5. Currency formatting: "50 ج.م" instead of "EGP 50"

13. Theme Engine — Storefront Customization

Overview

The theme engine is the bridge between the merchant dashboard and the customer-facing storefront. It implements a **Shopify-inspired section architecture** where storefronts are composed of configurable, reorderable sections.

Architecture Layers



```
| Reads customization data and renders |
```

Theme Schema System

Each theme defines a list of **SettingDefinition** objects:

```
// Example: colors, fonts, and layout settings
const settings: SettingDefinition[] = [
  { key: "primary_color", type: "color", label: "Primary Color", labelAr: "اللون الأساسي",
    default: "", group: "Colors" },
  { key: "heading_font", type: "font", label: "Heading Font", labelAr: "خط العناوين", default:
    "Cairo", options: fontOptions },
  { key: "border_radius", type: "range", label: "Border Radius", labelAr: "استدارة الحواف",
    default: 12, min: 0, max: 32, step: 2, unit: "px" },
  { key: "button_style", type: "select", label: "Button Style", labelAr: "شكل الزر", default:
    "rounded", options: styleOptions },
];
```

The dashboard reads these schemas and **dynamically generates** the settings form. No hardcoded forms — the UI adapts to whatever settings the theme declares.

Setting Types

Type	UI Control	Description
color	Color picker	RGB/hex color selection
text	Text input	Free-text string
select	Dropdown	Choose from predefined options
font	Font picker	Google Fonts with live preview
checkbox	Toggle switch	Boolean on/off
image	Image uploader	URL or file upload
range	Slider	Numeric value within min/max bounds

Section Engine (V2)

Sections are the building blocks of a storefront page:

```
// A store's template configuration
{
  order: ["hero-1", "featured-products-1", "testimonials-1"],
  sections: {
    "hero-1": {
      id: "hero-1",
      type: "hero-banner",
      settings: {
        heading: "Welcome to My Store",
      }
    }
  }
}
```

```

    subheading: "Discover amazing products",
    background_image: "https://...",
    cta_text: "Shop Now",
    cta_link: "/products"
  }
},
"featured-products-1": {
  id: "featured-products-1",
  type: "featured-products",
  settings: {
    title: "Best Sellers",
    count: 8,
    layout: "grid"
  }
}
}
}
}

```

Theme Editor Components

Component	Purpose
SchemaForm	Renders settings form from schema definitions
SettingControl	Individual setting control (color picker, slider, etc.)
SectionList	Ordered list of sections with drag-and-drop
SectionEditor	Configure individual section settings
AddSectionSheet	Slide-in panel to browse and add section types
CustomizationWalkthrough	First-time onboarding tour for theme editor

Draft → Publish Workflow

1. Merchant changes settings → `updateCustomization()` saves draft
2. Draft is **not** visible on the live storefront
3. Merchant clicks "Publish" → `publishCustomization()` makes it live
4. Storefront reads published configuration and renders accordingly

This prevents accidental publishing of half-finished designs.

14. Error Tracking & Observability

Sentry Integration

```

Sentry.init({
  dsn: import.meta.env.VITE_SENTRY_DSN,
  integrations: [
    Sentry.browserTracingIntegration(), // Performance tracing
    Sentry.replayIntegration(),         // Session replay for debugging
  ]
})

```



```

],
tracesSampleRate: 1.0,           // Trace 100% of transactions (dev rate)
replaysSessionSampleRate: 0.1,  // Record 10% of all sessions
replaysOnErrorSampleRate: 1.0,  // Record 100% of error sessions
});

```

What this captures:

- JavaScript errors with full stack traces
- HTTP request timing (browser tracing)
- DOM snapshots for error reproduction (session replay)
- Performance metrics: page load, route transitions, API call durations

Branch → Environment mapping (in CI):

Branch	Sentry Environment
dev	development
stage	staging
prod	production

Source Maps

The Sentry release workflow builds with `--sourcemap hidden` and uploads source maps to Sentry. This means:

- Source maps are **not** served to browsers (no reverse-engineering of minified code)
- Sentry **does** have source maps for readable stack traces
- Each release is tagged with the Git commit SHA

15. CI/CD Pipelines

Three GitHub Actions workflows automate quality assurance and deployment.

Pipeline 1: CI (`ci.yml`)

Triggered on push/PR to `dev` , `stage` , `prod` .

lint → test → build

Stage	Action
Lint	npm ci → npm run lint (ESLint)
Test	npm ci → npm run test (Vitest)
Build	npm ci → npm run build (TypeScript + Vite) → upload dist/ artifact

Concurrency: Only one CI run per branch at a time (`cancel-in-progress: true`).

Pipeline 2: Security (`security.yml`)

Triggered on push/PR to `dev` , `stage` , `prod` .

Stage	Action
NPM Audit	<code>npm audit --omit=dev --audit-level=high</code> — fails on high-severity vulnerabilities
Dependency Review	<code>dependency-review-action@v4</code> — PR comment summarizing new dependency risks

Pipeline 3: Sentry Release (`sentry-release.yml`)

Triggered **after CI succeeds** (via `workflow_run`).

Step	Action
Build with hidden source maps	<code>npx vite build --sourcemap hidden</code>
Create Sentry release	Tags with <code>numo-merchant-hub@{commit-sha}</code>
Upload source maps	Source maps sent to Sentry only
Record deployment	Maps branch to environment (dev/staging/production)

16. Security Considerations

Token Security

Aspect	Implementation	Risk Level
Token storage	<code>localStorage</code>	Medium — vulnerable to XSS but standard for SPAs
Token transmission	Bearer header over HTTPS	Low — standard practice
Token refresh	Automatic on 401, shared promise	Low — prevents race conditions
Token cleanup	Cleared on logout + failed refresh	Low
Cross-origin handoff	URL parameters, cleared immediately	Medium — token briefly in URL bar/history

Content Security

- **No** `dangerouslySetInnerHTML` in main code paths — reduces XSS attack surface
- **Zod validation** for form inputs — runtime type checking prevents malformed data
- **React's built-in XSS protection** — JSX auto-escapes rendered values
- `i18next escapeValue: false` — safe because React handles escaping at the rendering layer

Supply Chain Security

- **NPM Audit** runs on every push — catches vulnerable dependencies
- **Dependency Review** on PRs — flags new risky packages before merge
- **shadcn/ui components are vendored** — not pulled from npm at runtime

17. Environment Configuration

Environment Variables

Variable	Dev Value	Prod Value	Purpose
VITE_API_URL	http://localhost:8021/api/v1	https://api.numu.io/api/v1	Backend API base URL
VITE_STOREFRONT_URL	http://{subdomain}.localhost:8081	https://{subdomain}.numu.io	Storefront URL template
VITE_SENTRY_DSN	(optional)	Sentry project DSN	Error tracking

URL Template Pattern

The storefront URL uses a `{subdomain}` placeholder:

```
http://{subdomain}.localhost:8081 → http://my-shop.localhost:8081
https://{subdomain}.numu.io      → https://my-shop.numu.io
```

Helper functions in `src/lib/storefront.ts` :

- `getStoreUrl(subdomain)` — Builds full URL
- `getStoreDomainSuffix()` — Extracts suffix for display (`.numu.io` , `.localhost:8081`)

18. Local Development Setup

Prerequisites

- Node.js 20+** (specified in CI, compatible locally)
- npm** (lockfile is `bun.lockb` but npm works)
- NUMU API** running on `localhost:8021` (FastAPI backend)

Quick Start

```
# Clone and install
cd numo-merchant-hub
npm install

# Start dev server
npm run dev
# → http://localhost:8080

# Run tests
npm run test:watch

# Generate types from backend OpenAPI
npm run generate:types

# Build production bundle
npm run build
npm run preview    # preview at localhost:4173
```

Port Map

Service	Port	Protocol
Merchant Hub (this app)	8080	HTTP
NUMU API Backend	8021	HTTP
Storefront	8081	HTTP (wildcard subdomain)

19. Architecture Decisions & Trade-offs

Decision 1: React Query over Redux

Choice: TanStack React Query for server state. No Redux/Zustand.

Rationale: This is a dashboard where nearly all state comes from the server. React Query gives us caching, background refetch, stale-while-revalidate, optimistic updates, and query invalidation for free. Client-side state (auth, store selection, language) is small enough for Context.

Trade-off: No centralized client-side state store. If complex client-side logic emerges (e.g., multi-step wizard with cross-page state), Zustand may need to be added.

Decision 2: Custom `fetch` Wrapper over Axios

Choice: Hand-built `apiClient()` using native `fetch`.

Rationale: Zero dependency cost. The wrapper is ~60 lines. Axios would add 11KB for features we don't need (progress events, request cancellation, node.js support). The token refresh logic is the main complexity, and it's the same in either approach.

Trade-off: No built-in request interceptors, timeout handling, or upload progress. These would need to be added manually if needed.

Decision 3: shadcn/ui over Material UI / Ant Design

Choice: Vended Radix-based components via shadcn/ui.

Rationale: Full ownership of component code. No library-imposed design constraints. Smaller bundle (only import what you use, and it's just Tailwind classes). Perfect Tailwind integration. WAI-ARIA accessibility from Radix.

Trade-off: More manual work for complex components (data grid, date range picker, etc.). No out-of-the-box component library docs — team must read the source.

Decision 4: Adapter Pattern in Product Service

Choice: Three adapter functions between API and UI data shapes.

Rationale: Decouples frontend domain model from backend persistence model. The UI never knows about `attributes.name_ar` — it sees `nameAr`. If the API changes its response format, only the adapter changes.

Trade-off: Extra mapping code. Potential for bugs in mapping logic. Must keep adapters in sync with both API schema and UI type definitions.

Decision 5: Context over Props for Auth/Store/Language

Choice: Three React Context providers instead of prop drilling.

Rationale: Auth, current store, and language are cross-cutting concerns used by dozens of components. Prop drilling would be impractical. Context provides a clean global interface.

Trade-off: Any context value change re-renders all consumers. For auth and language this is fine (they change rarely). For store, it could cause unnecessary re-renders when switching stores, but React Query handles the expensive work (refetching data).

20. What I Would Change — Senior Perspective

After 20 years of building web applications, here's my honest assessment of what's strong and what could be improved:

What's Done Well

1. **Clean separation of concerns** — Services, contexts, pages, and components have clear boundaries
2. **Adapter pattern** — The product adapter layer is a mature engineering decision
3. **Token refresh deduplication** — The shared promise pattern prevents the classic multi-401 race condition
4. **RTL support** — Full bi-directional support, not an afterthought
5. **Theme schema engine** — Dynamic form generation from schema definitions is the right approach
6. **CI pipeline** — Lint → Test → Build → Security → Sentry release is a complete pipeline
7. **Draft/Publish workflow** — Prevents accidental theme deployments

What I Would Improve

1. **Error boundaries** — No `React.ErrorBoundary` components. A crash in one page component takes down the entire app. Add error boundaries around each route and around the layout.
2. **API error handling** — The api client throws errors, but many pages don't show error states. React Query provides `isError` and `error` — use them to show meaningful error UI instead of empty screens.
3. **Form validation** — Login and CreateStore use raw `useState` for forms. Products and Marketing use React Hook Form. Standardize on React Hook Form + Zod everywhere for consistency and better UX.
4. **Testing** — Vitest is configured but no test files are visible in the codebase. For a production app, critical paths need tests: auth flow, product adapter functions, order status transitions. These are the areas where bugs cost money.
5. **Token storage** — `localStorage` is the industry standard for SPAs, but `httpOnly` cookies would be more secure (immune to XSS). This would require backend changes to support cookie-based auth.
6. **URL token handoff** — Passing tokens in URL parameters works but leaves tokens in browser history. Consider using a short-lived, one-time authorization code (OAuth-style) that gets exchanged for tokens.
7. **Loading states** — Some pages show the `NumuLoadingScreen` during auth, but data loading in pages uses skeleton placeholders inconsistently. Standardize the loading UX.
8. **COD & Social Import** — Both use mock data. These features should either be completed with real API integration or clearly marked as "coming soon" in the UI.
9. **Bundle analysis** — No webpack-bundle-analyzer or rollup-plugin-visualizer configured. At 30+ Radix packages, it's worth checking the tree-shaking is working correctly.
10. **Accessibility audit** — Radix provides great baseline a11y, but custom components (Dashboard charts, theme editor) need manual a11y testing. Color contrast in both light and dark modes should be verified.

Appendix A: Dependency Count Summary

Category	Count
Radix UI primitives	30 packages
Core React ecosystem	6 packages
Build/dev tools	12 packages
UI utilities	8 packages
Data/state libraries	4 packages
i18n	3 packages
Observability	1 package
Total production deps	~50
Total dev deps	~20

Appendix B: API Endpoint Inventory

Service	Endpoints	Methods
Auth	4	POST, GET
Stores	9	GET, POST, PUT
Products	7	GET, POST, PUT, DELETE
Orders	7	GET, POST, PUT
Customers	2	GET
Analytics	9	GET
Categories	4	GET, POST, PUT, DELETE
Coupons	5	GET, POST, PUT, DELETE
Themes	5	GET, PUT, POST
Total	52 endpoints	

Appendix C: Route Map

Path	Component	Auth	Store	Layout
/login	Login	No	No	None
/create-store	CreateStore	Yes	No	None

/	Dashboard	Yes	Yes	DashboardLayout
/products	Products	Yes	Yes	DashboardLayout
/orders	Orders	Yes	Yes	DashboardLayout
/store	StoreSettings	Yes	Yes	DashboardLayout
/cod	CODReconciliation	Yes	Yes	DashboardLayout
/social	SocialImport	Yes	Yes	DashboardLayout
/customers	Customers	Yes	Yes	DashboardLayout
/analytics	Analytics	Yes	Yes	DashboardLayout
/marketing	Marketing	Yes	Yes	DashboardLayout
/categories	Categories	Yes	Yes	DashboardLayout
*	NotFound	No	No	None

This document was generated through comprehensive source code analysis of all 42+ source files in the numo-merchant-hub repository.